

Estruturas do Sistema Operacional

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Serviços do sistema operacional
- Interface do SO com usuário
- Chamadas ao sistema
- Programas de sistema
- Projeto e implementação de SOs
- Estrutura do SO
- Máquinas virtuais
- Depuração, geração e boot do SO

Serviços do sistema operacional

- Serviços providos pelo SO: para usuários e para eficiência do próprio SO
 - Variam muito de SO para SO
 - Também visam tornar a programação mais simples: APIs
 - Comumente...

Serviços do sistema operacional

- Serviços que provêem funções úteis ao usuário:
 - Interface com usuário: todos SOs têm uma UI
 - Pode ser via linha de comando (CLI) ou gráfica (GUI)
 - Execução de programas: SO deve ser capaz de carregar programa na memória, executar e terminar execução (normalmente/erro)
 - Operações de E/S: necessárias para executar arquivos ou acessar dispositivo de E/S
 - Usuário não pode controlar dispositivos de E/S diretamente
 - Manipulação de sistemas de arquivos: possibilidade de ler/escrever arquivos, criar/apagar/buscar arquivos e diretórios via API

Serviços do sistema operacional

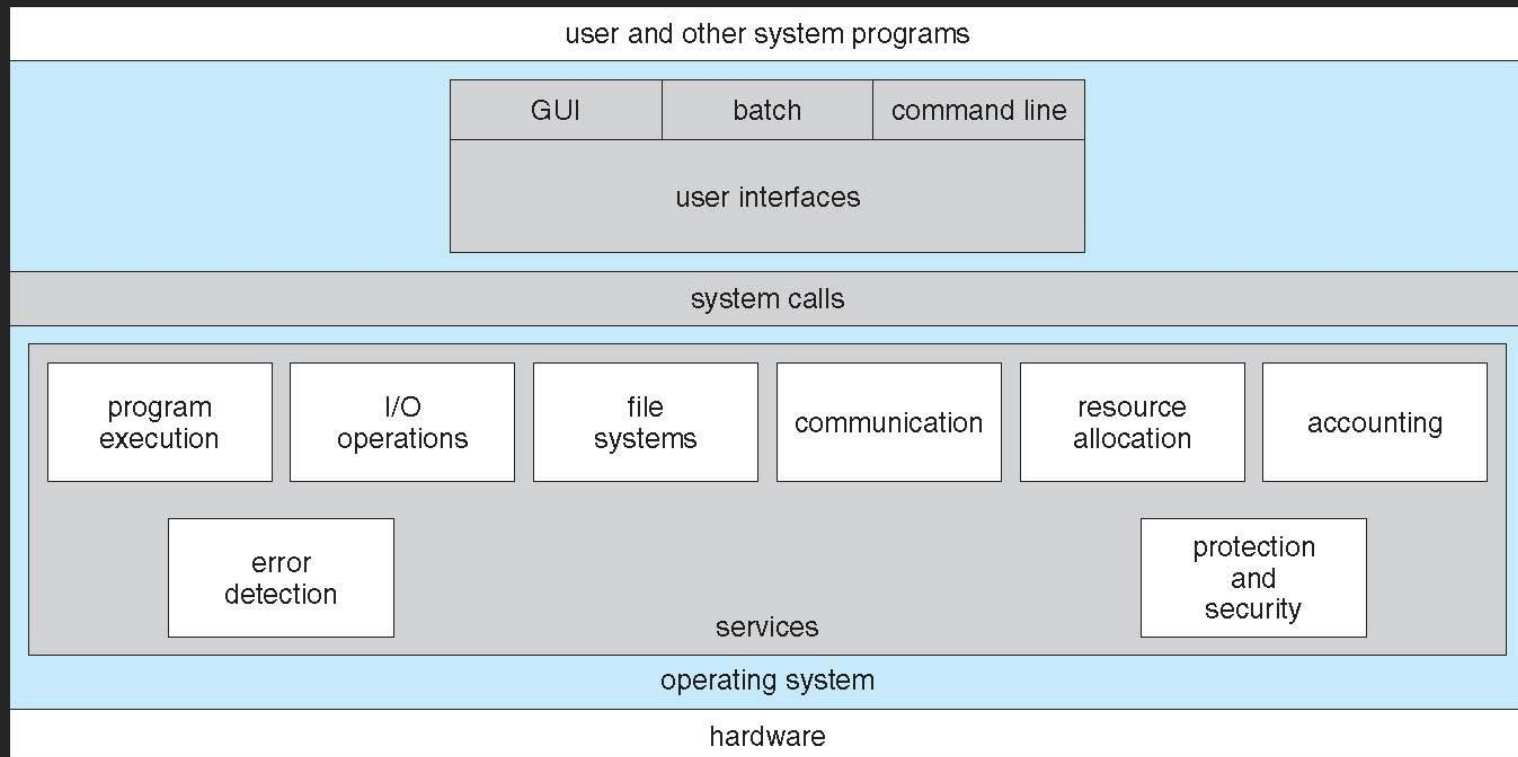
- (cont.)
 - Comunicação: processos necessitam trocar mensagens no mesmo computador ou externamente
 - Mecanismos: memória compartilhada, passagem de mensagens
 - Detecção de erro: em hardware ou software
 - Ex.: hardware da CPU e memória, dispositivos de E/S (erro de paridade, falha de conexão impressora), em processos (overflow aritmético, acesso ilegal à memória)
 - SO provê: formas de contornar erros e meios de depuração

Serviços do sistema operacional

- Serviços que existem para garantir eficiência do sistema:
 - Alocação de recursos: múltiplos usuários e processos concorrendo
 - Recursos diversos devem ser alocados a cada um: CPU, memória, dispositivos de E/S
 - Contabilidade: contabilizar quais usuários/processos usam que quantidade de quais recursos
 - Estatísticas mantidas para, por exemplo, cobrar usuários
 - Úteis para reconfigurar sistema
 - Proteção e segurança: controlar acesso aos recursos (proteção) e defesa contra ataques
 - “Uma corrente é tão forte quanto seu elo mais fraco”

Serviços do sistema operacional

- Serviços do SO:



Interface do SO com usuário

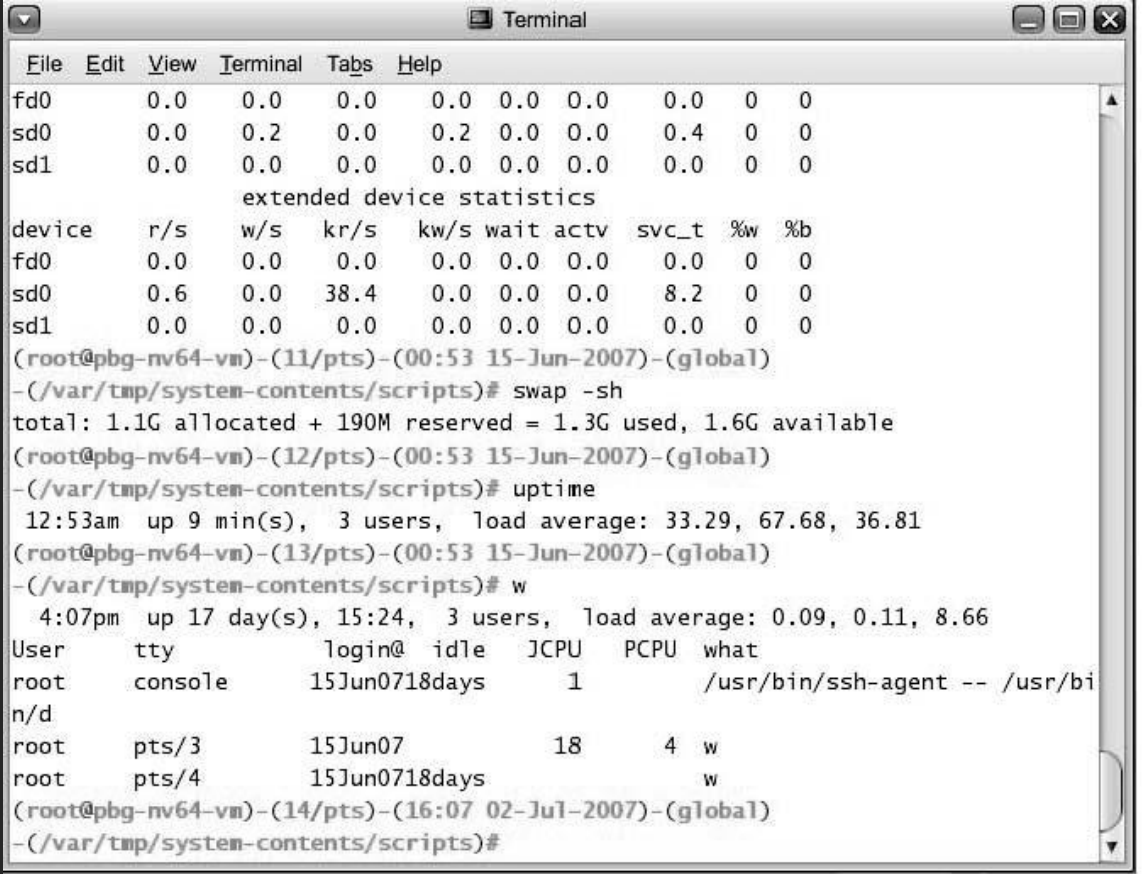
- Duas abordagens básicas: linha de comando (ou interpretador de comandos) e interface gráfica
- Interpretador de comandos: incluso no kernel por alguns SOs ou incluso como um programa especial (UNIX, Windows)
 - Utilidade: receber e executar próximo comando do usuário
 - Comandos: relacionados à manipulação de arquivos e execução de programas
 - Um SO pode ter diversos interpretadores de comandos (shells): funcionalidades similares

Interface do SO com usuário

- Execução de comandos: duas formas
 - Embutidos no interpretador de comandos: comando (ex.: deletar arquivo) é codificado no próprio interpretador
 - Complexidade do interpretador: quantidade/dificuldade dos comandos
 - Comandos não são implementados no interpretador
 - Interpretador não conhece: somente carrega arquivos para execução
 - Ex.: rm do UNIX

Interface do SO com usuário

- Bourne Shell (sh):



```
Terminal
File Edit View Terminal Tabs Help
fd0      0.0    0.0    0.0    0.0 0.0 0.0 0.0 0.0 0 0
sd0      0.0    0.2    0.0    0.2 0.0 0.0 0.0 0.4 0 0
sd1      0.0    0.0    0.0    0.0 0.0 0.0 0.0 0.0 0 0
extended device statistics
device   r/s    w/s    kr/s    kw/s wait actv  svc_t  %w  %b
fd0      0.0    0.0    0.0    0.0 0.0 0.0  0.0  0  0
sd0      0.6    0.0   38.4    0.0 0.0 0.0  8.2  0  0
sd1      0.0    0.0    0.0    0.0 0.0 0.0  0.0  0  0
(root@pbg-nv64-vm)-(11/pts)-(00:53 15-Jun-2007)-(global)
-(/var/tmp/system-contents/scripts)# swap -sh
total: 1.1G allocated + 190M reserved = 1.3G used, 1.6G available
(root@pbg-nv64-vm)-(12/pts)-(00:53 15-Jun-2007)-(global)
-(/var/tmp/system-contents/scripts)# uptime
12:53am up 9 min(s), 3 users, load average: 33.29, 67.68, 36.81
(root@pbg-nv64-vm)-(13/pts)-(00:53 15-Jun-2007)-(global)
-(/var/tmp/system-contents/scripts)# w
4:07pm up 17 day(s), 15:24, 3 users, load average: 0.09, 0.11, 8.66
User      tty          login@ idle   JCPU   PCPU   what
root      console      15Jun0718days    1          /usr/bin/ssh-agent -- /usr/bi
n/d
root      pts/3        15Jun07          18     4    w
root      pts/4        15Jun0718days          w
(root@pbg-nv64-vm)-(14/pts)-(16:07 02-Jul-2007)-(global)
-(/var/tmp/system-contents/scripts)#
```

Interface do SO com usuário

- Interfaces gráficas: metáfora desktop (mesa de trabalho)
 - Intuitiva: user-friendly
 - Inventada na Xerox PARC: anos 70
 - Difundida pela Apple: anos 80
 - Windows 1.0: interface para o MS-DOS em 1985
 - Ícones representam arquivos, programas, ações, etc.
 - Baseada no uso do mouse
 - Botões do mouse pressionados sobre objetos da interface disparam diversas ações: mostrar informação/opções, executar programas, abrir diretórios (pasta/folder), alterar arquivos, etc.

Interface do SO com usuário

- GUI do MAC OS X:

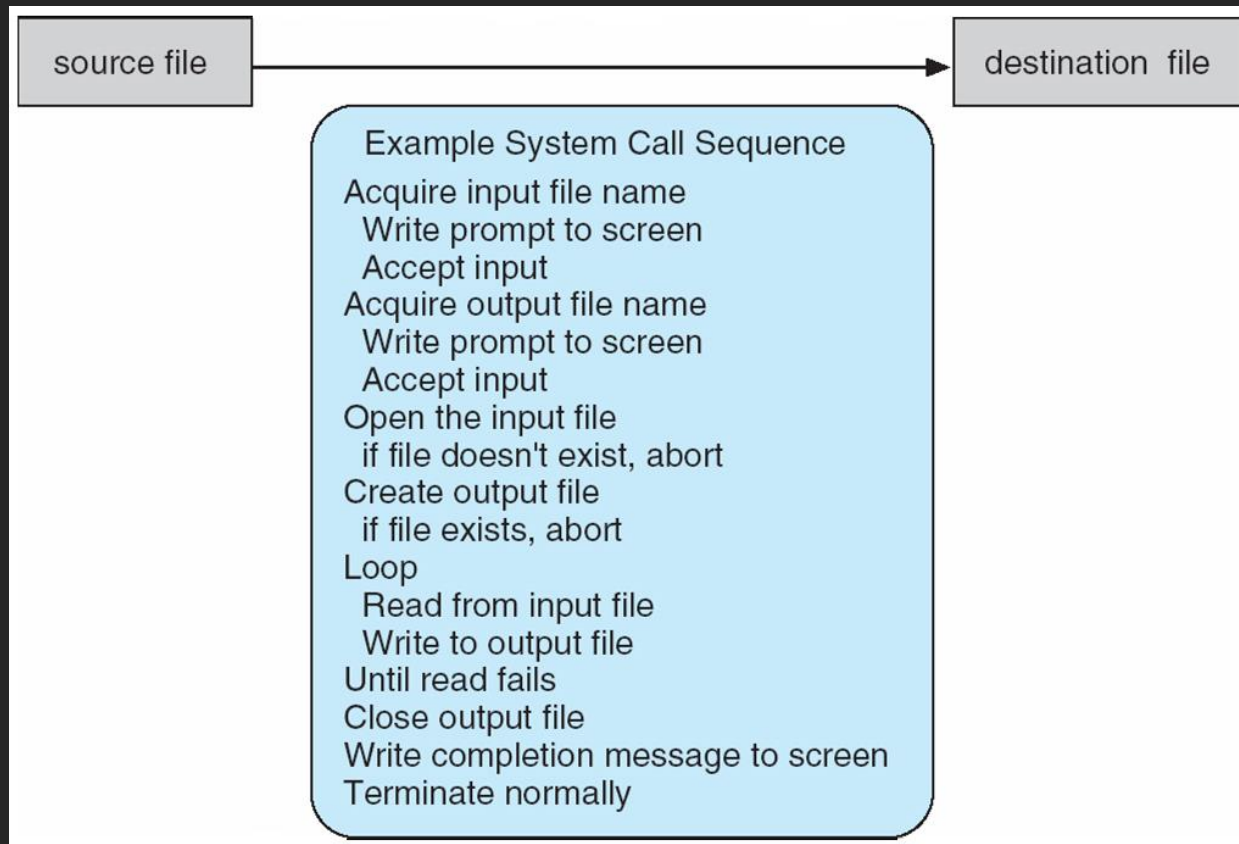


Chamadas ao sistema

- “As chamadas ao sistema (system calls) provêem uma interface aos serviços disponibilizados por um sistema operacional”
 - Interface de programação disponibilizada pelo SO
 - Geralmente implementadas em alto nível (C/C++)
 - Programas simples fazem milhares de chamadas ao SO por segundo (ex. no próx. slide)
 - Invocação em geral via APIs de alto nível (ex.: open /fopen)
 - Principais: Win32 API, POSIX API (UNIX, Linux e MAC OS X) e Java API

Chamadas ao sistema

- Chamadas ao sistema feitas para copiar arquivo:

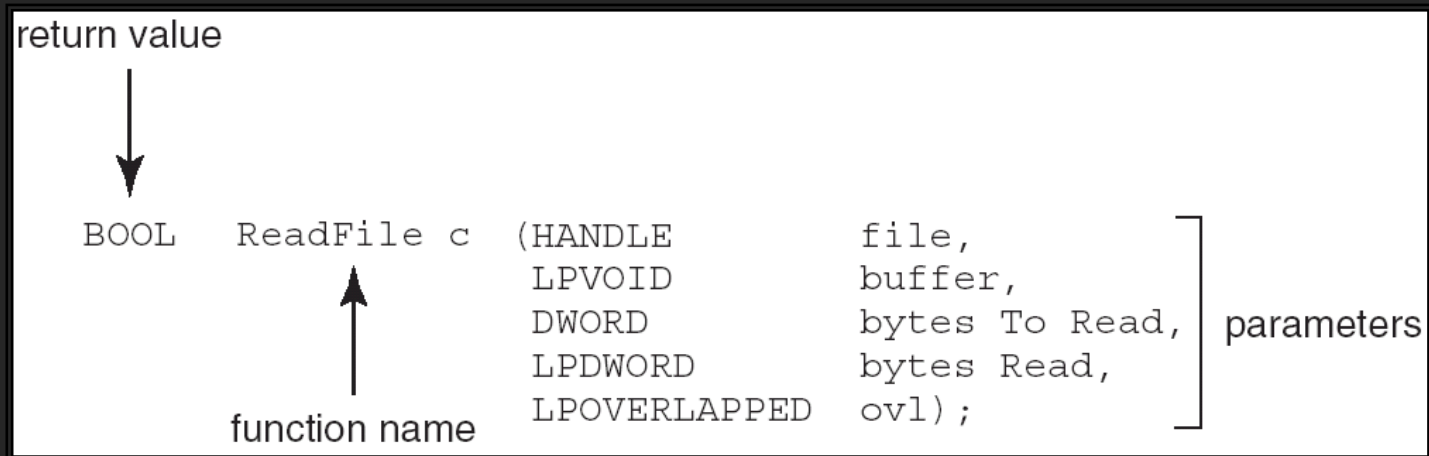


Chamadas ao sistema

- Porque usar APIs ao invés de fazer a chamada ao SO?
 - Portabilidade: programa pode ser compilado em qualquer sistema que suporte API
 - Chamadas ao SO podem ser mais detalhadas e difíceis de lidar
 - Mas existe forte correlação entre uma função da API e a chamada ao SO correspondente

Chamadas ao sistema

- Exemplo de função da API Win32: ReadFile()

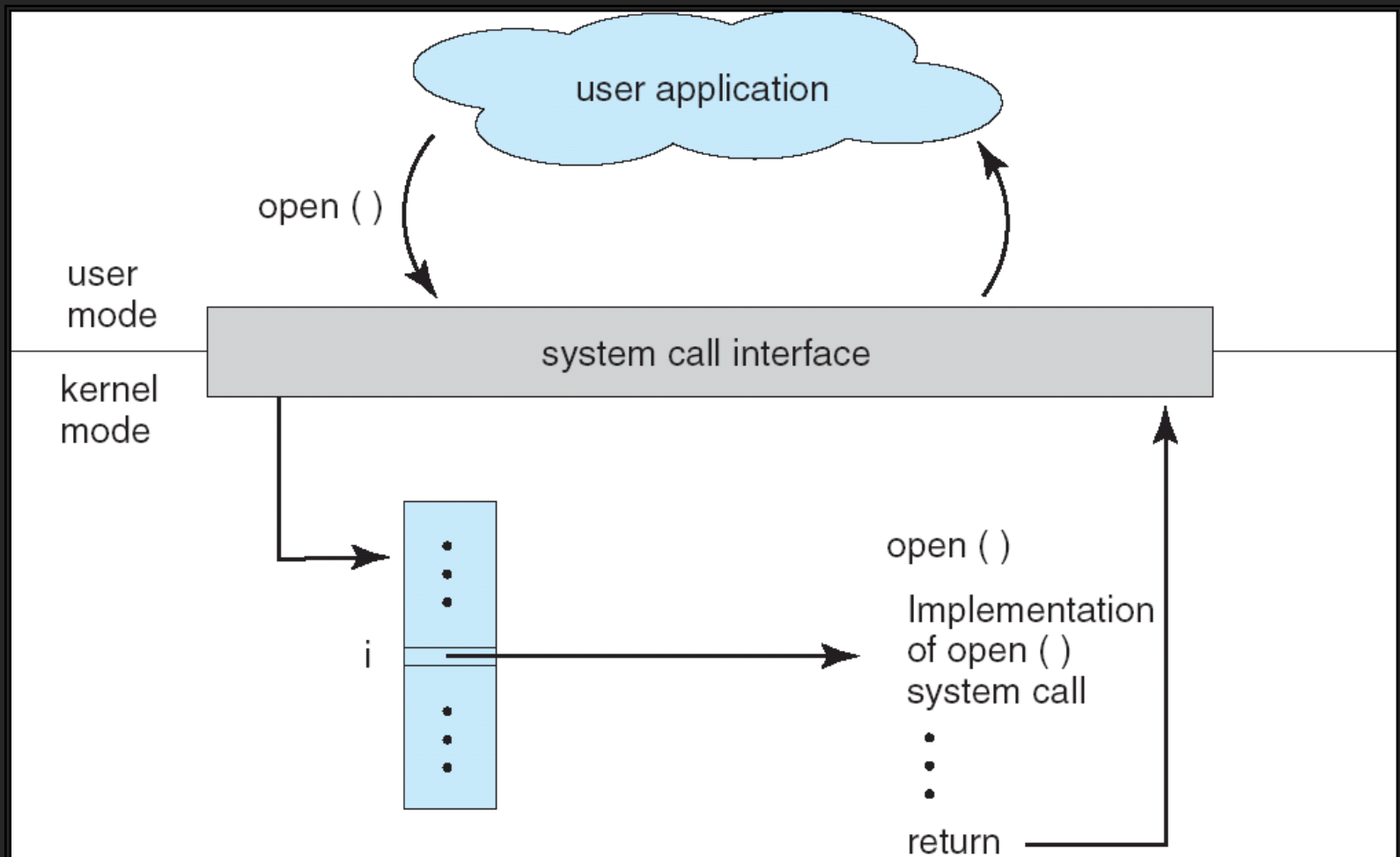


- `HANDLE file`: arquivo a ser lido
- `LPVOID buffer`: buffer para leitura
- `DWORD bytesToRead`: número de bytes a serem lidos e copiados para o buffer
- `LPDWORD bytesRead`: número de bytes lidos anteriormente
- `LPOVERLAPPED ovl`: indica se há sobreposição de operação de E/S

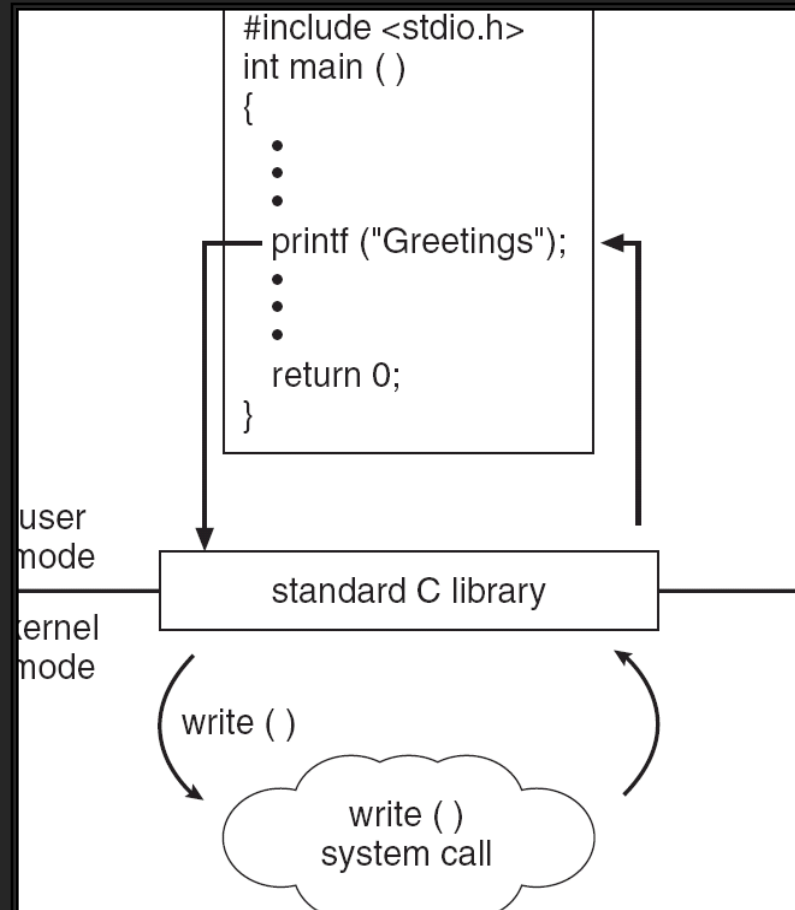
Chamadas ao sistema

- Implementação de chamadas ao SO:
 - Bibliotecas run-time (compilador) fornecem uma interface para as chamadas ao sistema
 - Ex.: `syscall()` declarada em `sys/syscall.h`
 - Um número é associado a cada chamada implementada
 - SO mantém uma tabela indexada por esses números
 - A interface invoca a chamada ao SO e retorna status/valores de retorno dessa chamada
 - Programador não precisa saber como chamada é implementada: somente segue API do sistema

Chamadas ao sistema



Chamadas ao sistema

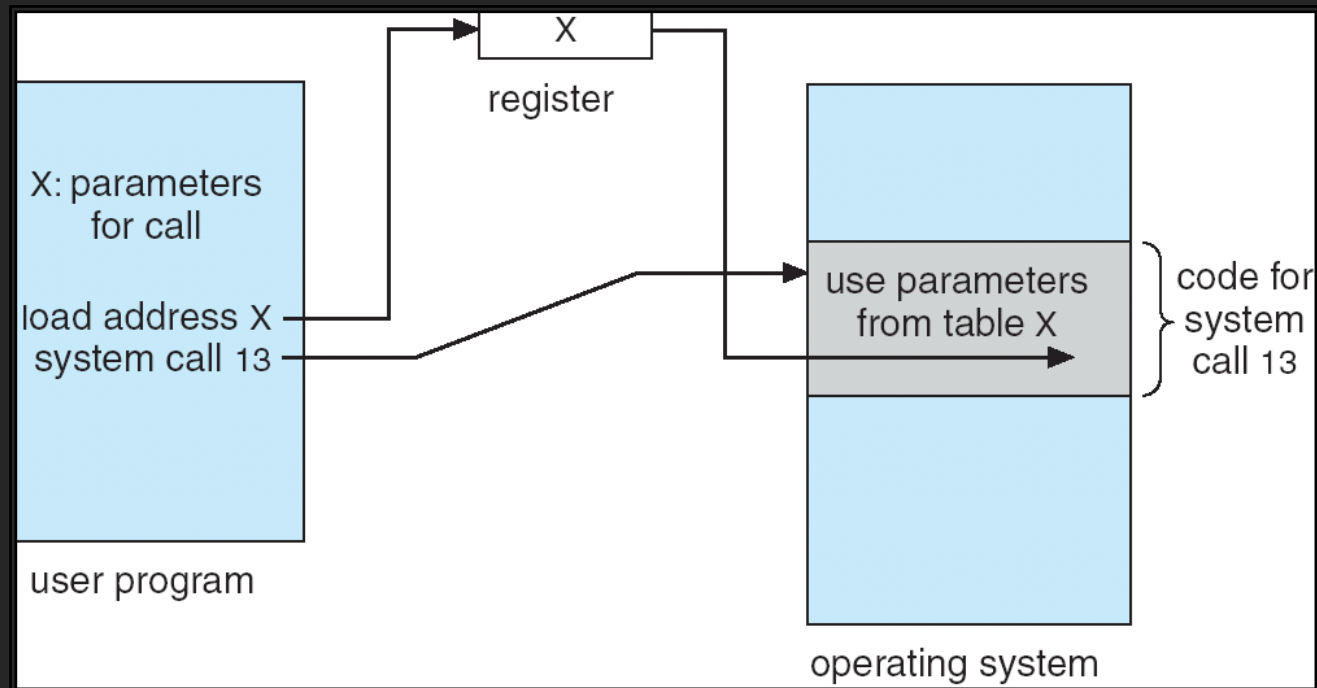


Chamadas ao sistema

- Chamadas ao SO podem exigir zero ou mais parâmetros
 - Ex.: getpid, exit, open, write
- Passagem de parâmetros às chamadas ao SO:
 - Parâmetros armazenados em registradores: mais simples
 - Porém, limita quantidade/tamanho de parâmetros
 - Parâmetros armazenados em tabela na memória: registrador é usado para indicar endereço da tabela
 - Linux e Solaris
 - Parâmetros são empilhados antes da chamada e desempilhados pelo SO

Chamadas ao sistema

- Passagem de parâmetros através de tabela:



Chamadas ao sistema

- Tipos de chamadas ao SO:
 - Controle de processo:
 - Criar, carregar, executar, terminar, abortar, ler/escrever atributos, esperar por tempo/evento, sinalizar evento, alocar e liberar memória
 - Gerenciamento de arquivos:
 - Criar e apagar arquivos/diretórios, abrir, fechar, ler, escrever, reposicionar, ler/escrever atributos
 - Gerenciamento de dispositivos:
 - Requisitar e liberar dispositivo, ler, escrever, reposicionar, ler/escrever atributos, conectar/desconectar logicamente dispositivos

Chamadas ao sistema

- (cont.):
 - Manutenção de informação:
 - Ler/alterar hora/data
 - Ler/escrever atributos de processos/arquivos/dispositivos
 - Comunicação:
 - Criar/terminar conexão
 - Enviar/receber mensagens
 - Ler informações de status de transferência
 - Conectar/desconectar dispositivos remotos
 - Proteção:
 - Ler/alterar permissões de arquivos/dispositivos, permitir/negar acesso a recursos

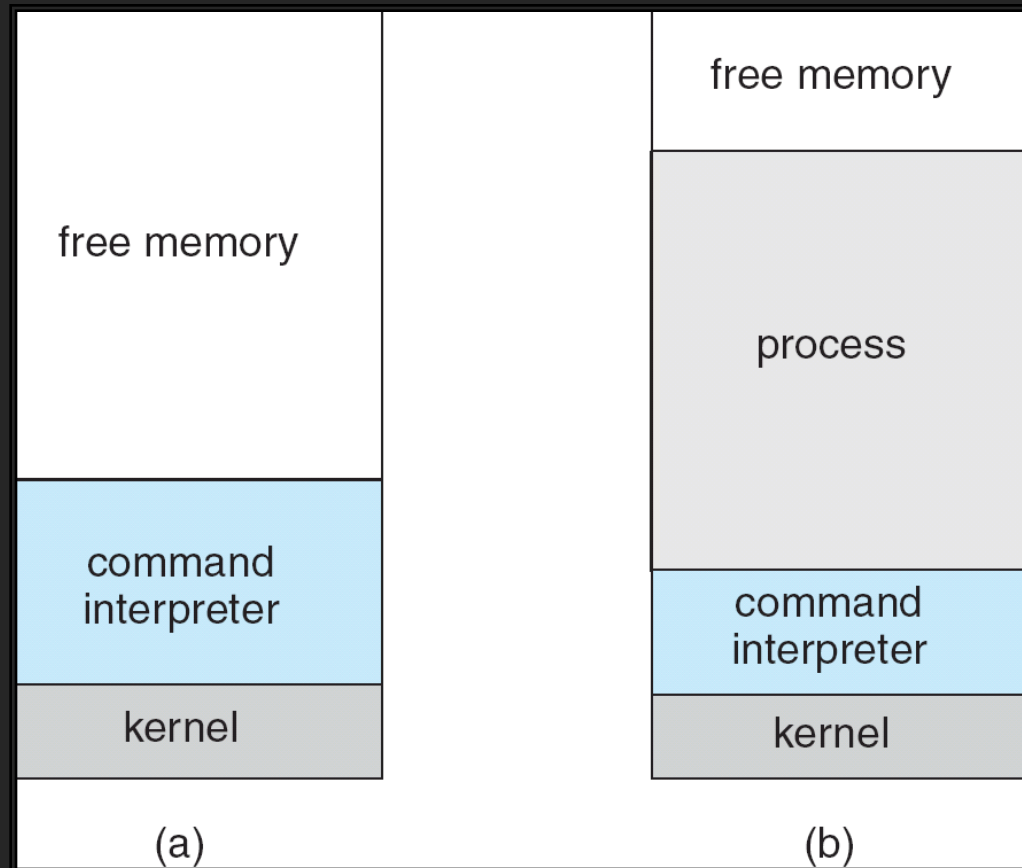
Chamadas ao sistema

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Chamadas ao sistema

- Execução de processos no MS-DOS (monotarefa):
 - A cada nova tarefa não necessita criar novo processo: apenas sobrescreve anterior
 - Parte do SO (interpretador de comandos) também é sobrescrita: mais memória disponível
 - No término da tarefa, o restante do interpretador é recarregado na memória

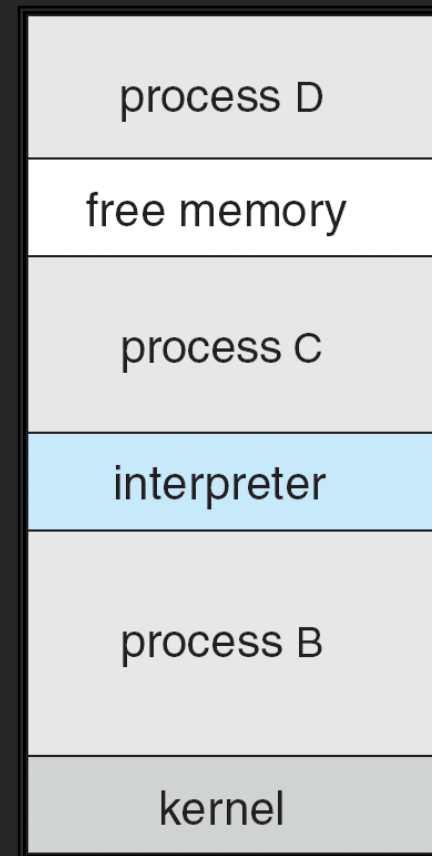
Chamadas ao sistema



(a) At system startup; (b) running a program

Chamadas ao sistema

- FreeBSD (multitarefa): necessita chamadas ao SO mais elaboradas
 - Shell realiza `fork()`
 - Programa é carregado: `exec()`



Programas de sistema

- “Programas de sistema (utilitários do sistema) provêem ambiente conveniente para execução/desenvolvimento de programas”
 - Tipos: manipulação de arquivos, informações do sistema, edição de arquivos, linguagem de programação, carregamento e execução de programas, comunicação
- Visão do SO por muitos usuários: definida pelos programas de sistema, não pelas chamadas ao SO

Programas de sistema

- Manipulação de arquivos:
 - Criar, apagar, copiar, renomear, imprimir, pesquisar, listar arquivos/diretórios
- Informações do sistema:
 - Saída formatada/impressão: data, hora, memória disponível, espaço em disco, número de usuários, versão do sistema, informações de log e depuração
- Edição de arquivos:
 - Editores de texto, programas para buscar/comparar/alterar arquivos

Programas de sistema

- Linguagem de programação:
 - Compiladores, montadores, depuradores, interpretadores
- Carregamento e execução de programas:
 - Carregadores absolutos/relocáveis, link editores, carregadores de overlays, depuradores
- Comunicação:
 - Envio/recebimento de mensagens para usuários, abrir páginas web, enviar/recebimento de e-mails, login remoto, transferência de arquivos

Projeto e implementação de SOs

- Projeto e implementação de SOs: algumas abordagens de sucesso
 - Estrutura interna de SOs podem variar muito
 - Início: definir metas e especificações
 - Tipo do hardware? Tipo do sistema?
 - Ex.: VxWorks (SO de tempo real para sistemas embarcados) e MVS (largamente multiusuário/multiacesso para mainframes)
 - Metas de usuário x metas do sistema
 - Metas de usuário: SO deve ser conveniente para usar, fácil de aprender, confiável, seguro, rápido
 - Metas do sistema: SO deve ser fácil de projetar/implementar/manter, flexível, confiável, livre de erros, eficiente

Projeto e implementação de SOs

- Princípios importantes a serem separados
 - Política: o que será feito?
 - Ex.: justiça entre usuários competindo pela CPU
 - Mecanismo: como será feito?
 - Ex.: implementação do algoritmo de escalonamento Round-Robin
 - A separação de política/mecanismo permite flexibilidade se decisões de política são alteradas posteriormente

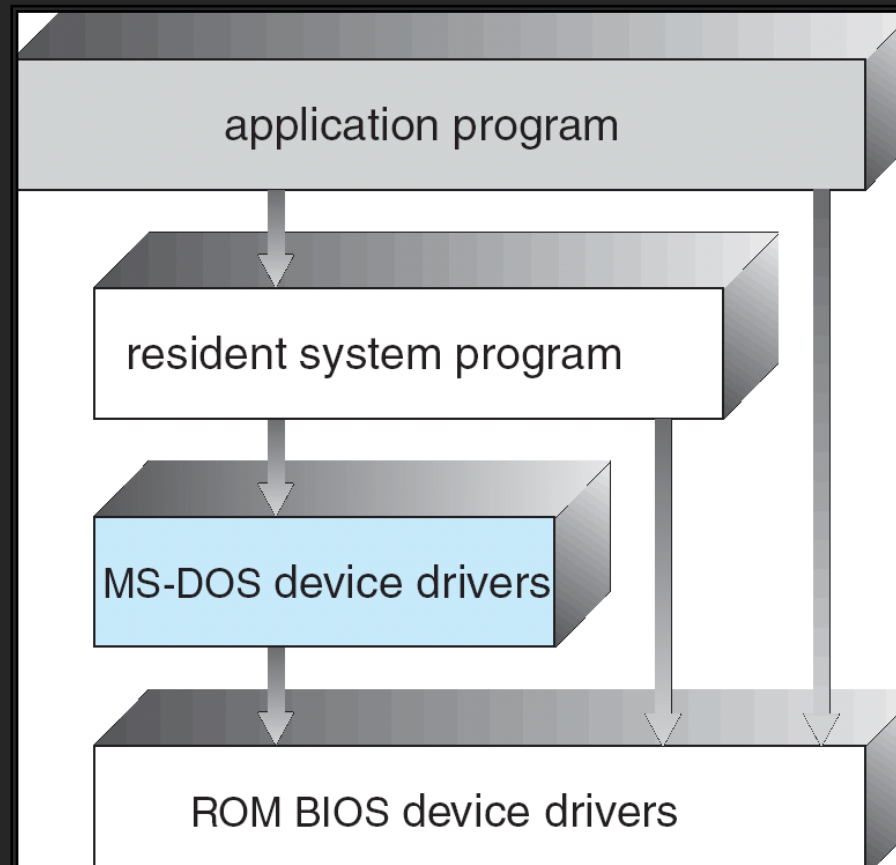
Estrutura do SO

- SO: sistema grande e complexo
 - Projeto deve ser cuidadoso: funcionar e ser modificado facilmente
 - Abordagem comum: componentes/módulos bem definidos, ao invés de sistema monolítico
- Estrutura do SO: define como componentes são separados e interconectados

Estrutura do SO

- Estrutura simples: MS-DOS
 - Inicialmente pequeno, simples, limitado
 - Hardware Intel 8088: sem modo dual
 - Meta: ocupar pouco espaço
 - Sem módulos bem definidos
 - Programas de aplicação podiam acessar diretamente dispositivos de E/S
 - Liberdade para escrever aplicações mais rápidas
 - Programas com erros/maliciosos: crash do sistema

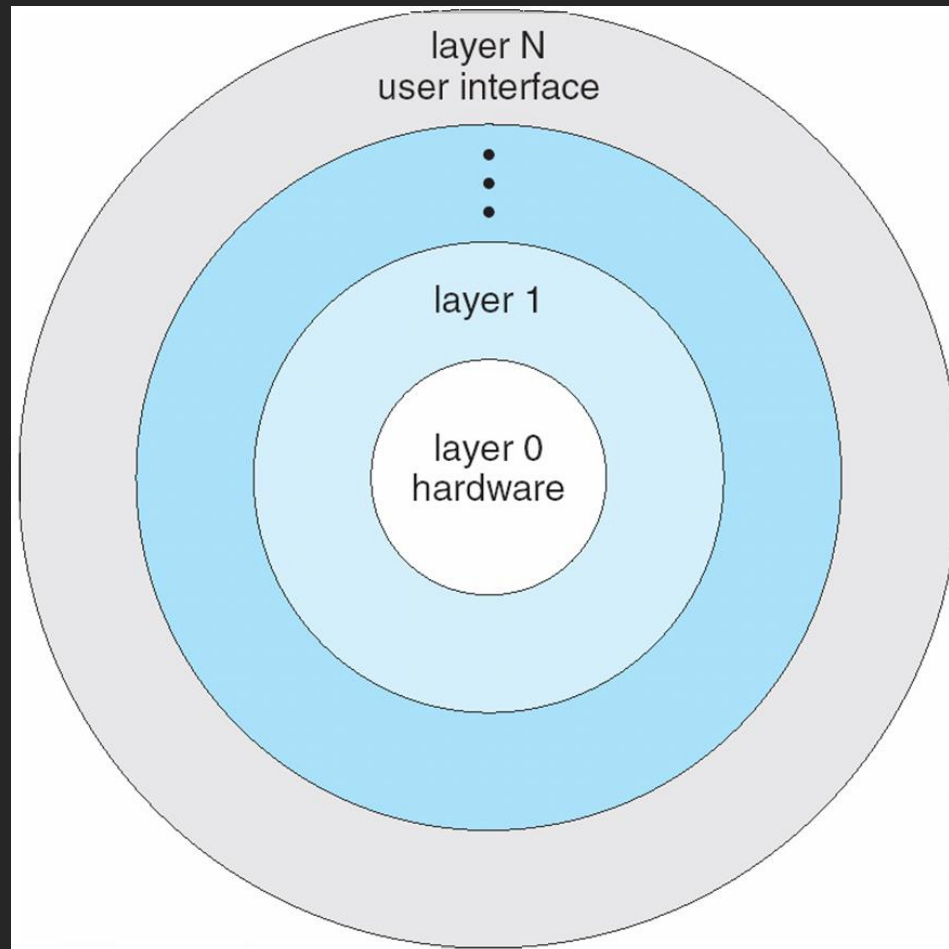
Estrutura do SO



Estrutura do SO

- Abordagem em camadas:
 - SO dividido em diversas camadas (níveis): camada superior construída sobre as inferiores (e disponibiliza funções para camadas superiores)
 - Camada 0: hardware
 - Camada N: interface com usuário
- Vantagens:
 - Mais controle sobre computador e aplicações
 - Mais fácil de construir e depurar
- Necessário suporte de hardware

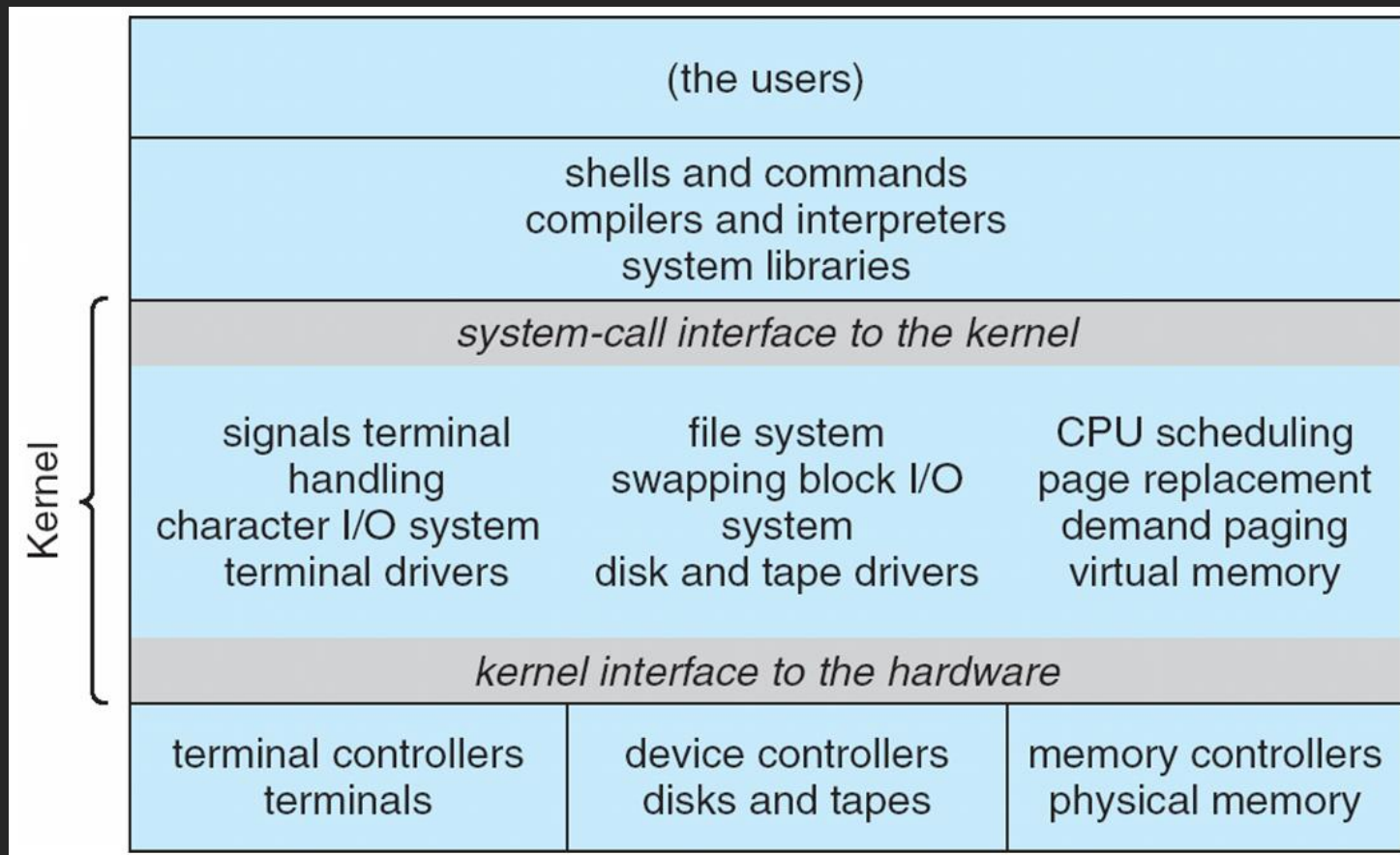
Estrutura do SO



Estrutura do SO

- UNIX: limitado pela funcionalidade do hardware
 - UNIX original: estrutura limitada (dois níveis)
 - Programas de sistema
 - Kernel: tudo abaixo da interface chamadas ao sistema e acima do hardware
 - Sistema de arquivos, escalonamento de CPU, gerenciamento de memória, etc.: um número muito grande de funções para um único nível

Estrutura do SO



Estrutura do SO

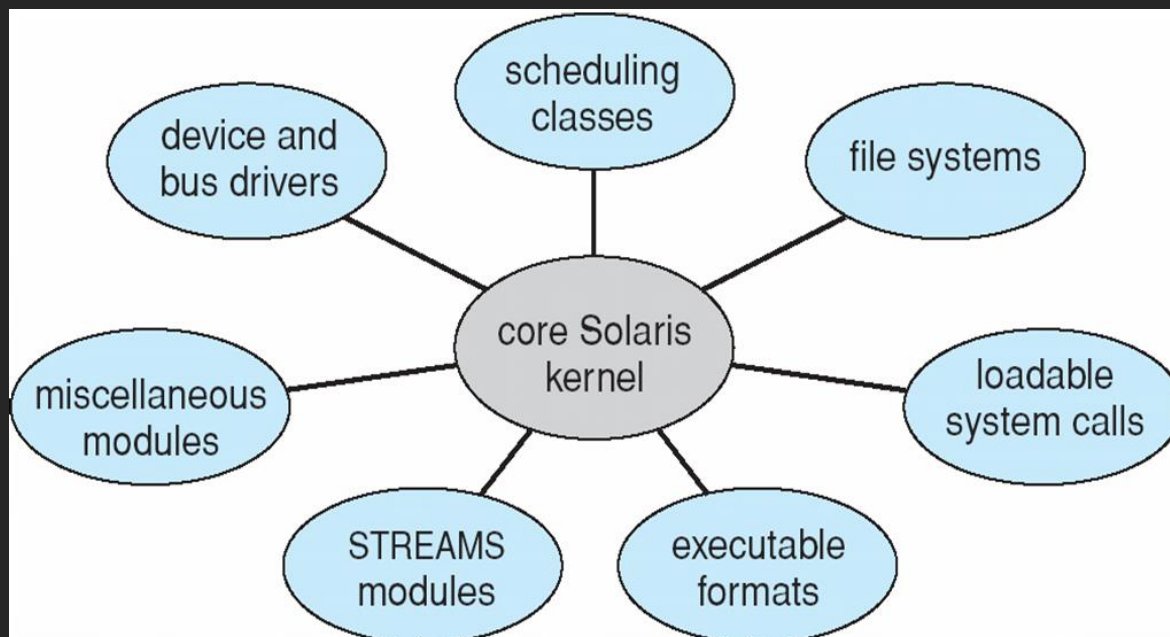
- Microkernel: move o máximo do kernel para espaço do “usuário”
 - Kernel: ger. memória + processos + comunicação
 - Comunicação entre componentes: passagem de mensagens
 - Vantagens:
 - Mais fácil de desenvolver, estender e portar
 - Código mais confiável (menos código no modo kernel)
 - Falha em componente: impacto é limitado
 - Mais seguro
 - Desvantagens: baixo desempenho
 - Exemplos: Mach, Hurd

Estrutura do SO

- Muitos SOs modernos implementam módulos de kernel
 - Usa abordagem da programação orientada a objetos
 - Kernel: conjunto de componentes de núcleo
 - Cada componente de núcleo é separado
 - Comunicação direta através de interfaces conhecidas
 - Carregados dinamicamente quando necessários
 - Ex.: sistema de arquivos DVD
 - Similar à:
 - Abordagem em camadas: porém, mais flexível
 - Componente pode chamar qualquer outro
 - Microkernel (essencial em módulo primário): porém, mais eficiente (sem troca de mensagens)

Estrutura do SO

- Implementações modernas do UNIX usam estrutura de módulos de kernel: Solaris, Linux, Mac OS X
- Módulos do Solaris:

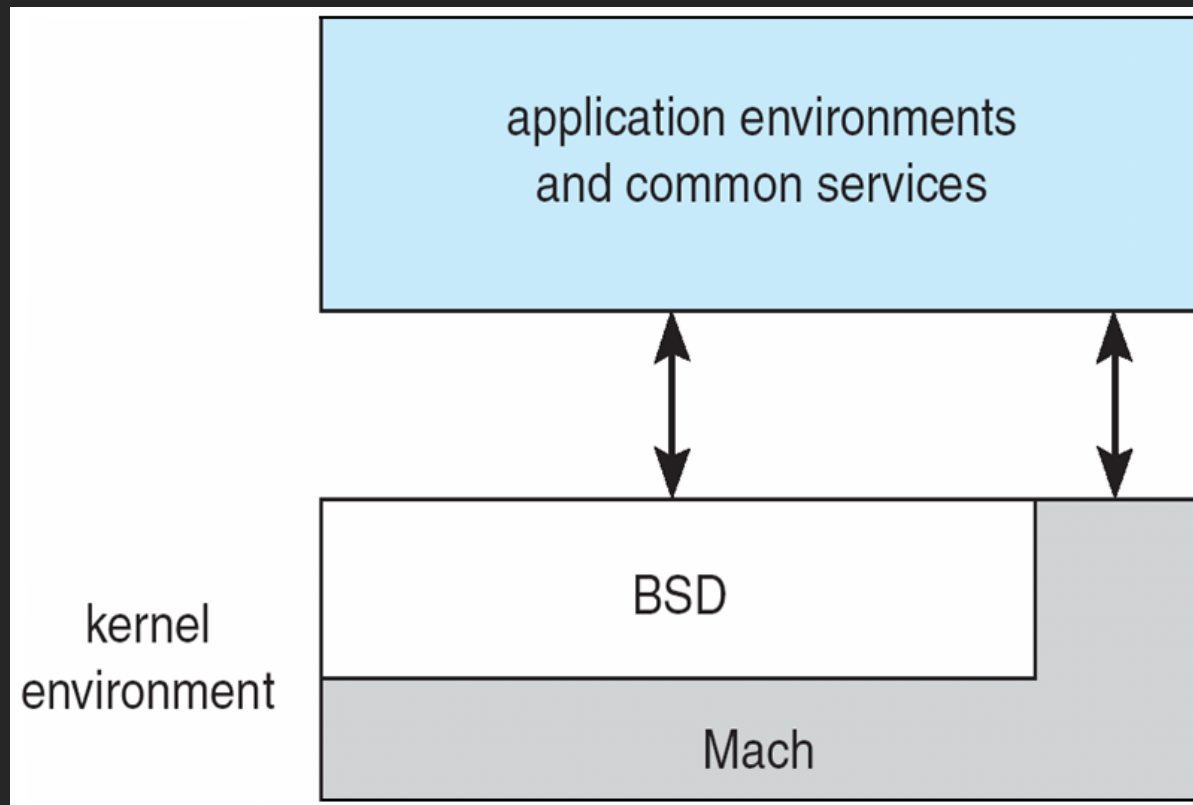


Estrutura do SO

- Mach OS X: abordagem híbrida
 - Em camadas: uma camada é o micronúcleo Mach
 - Camada superior: interface gráfica
 - Camada inferior: Mach + BSD kernels
 - Mach: ger. memória, IPC, RPC, escalonamento de threads
 - BSD: linha de comando, rede, sistemas de arquivo, POSIX APIs

Estrutura do SO

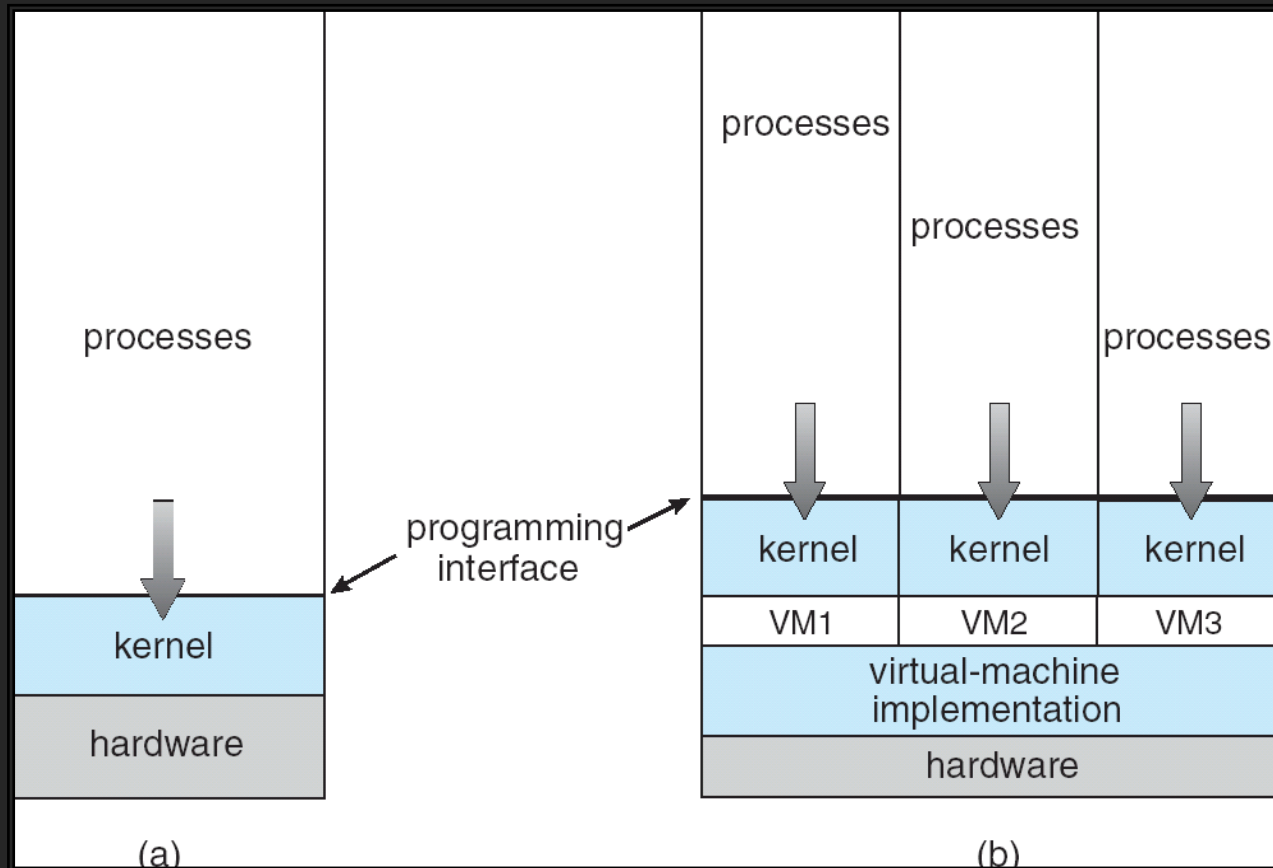
- Estrutura do Mach OS X:



Máquinas virtuais

- Máquina virtual (MV): abstrai hardware em diversos ambientes de execução
 - MV provê interface idêntica ao hardware básico
 - SO hospedeiro cria ilusão de que cada processo possui processador e memória
- Virtualização: SO compilado para determinada CPU executa dentro de outro SO nativo para mesma CPU
- Para-virtualização: SO convidado (modificado) recebe ambiente similar (não idêntico)
- Emulação/simulação: possibilita executar programas para outras arquiteturas

Máquinas virtuais



(a) Nonvirtual machine; (b) virtual machine

Máquinas virtuais

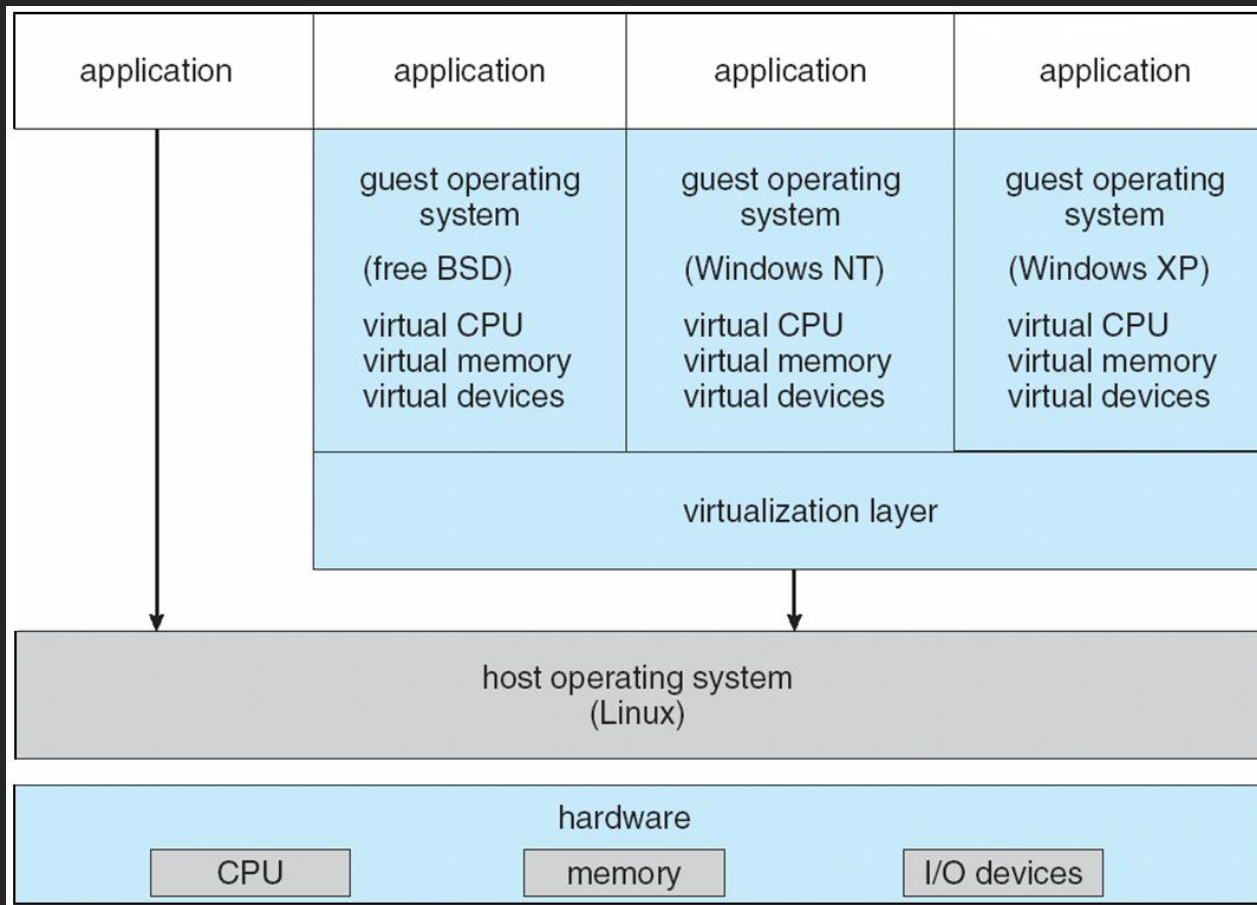
- Vantagens:
 - Compartilhamento de hardware
 - Uso de diferentes ambientes de execução
 - Isolamento: proteção do sistema hospedeiro e das máquinas virtuais de problemas em uma máquina virtual
 - Ex.: vírus
 - Desenvolvimento/pesquisa de SOs
 - Consolidação: junção de dois sistemas em mesmo sistema
- Dificuldades:
 - Implementação, isolamento, desempenho

Máquinas virtuais

- Implementação: suporte especial de hardware
 - Máquina real: modo usuário e sistema
 - Máquina virtual: modo usuário virtual e sistema virtual
- Ex.: Chamada ao sistema feita por programa executando em modo usuário virtual
 - Tratamento feito pelo monitor da máquina virtual em execução na máquina real
 - Simulação da execução
 - Impacto: E/S pode ser mais rápida ou lenta que real
 - Muitos fatores a analisar: imprevisível

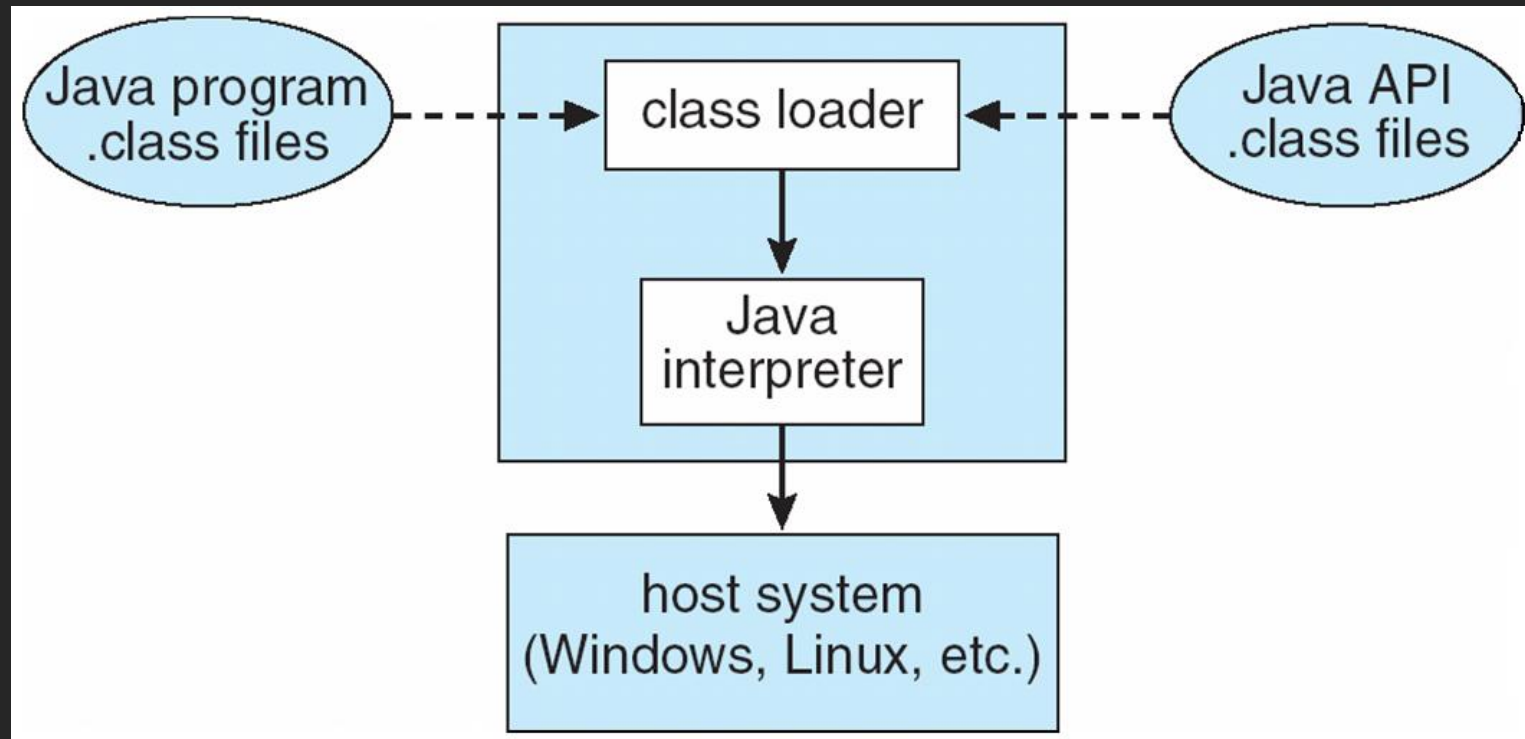
Máquinas virtuais

- Arquitetura da VMware:



Máquinas virtuais

- Máquina virtual Java:



Depuração, geração e boot do SO

- Depuração: encontrar e consertar erros
 - Inclusive de desempenho: gargalos
 - Falha em aplicação: arquivo core dump
 - Memória do processo
 - Falha no SO: arquivo crash dump
 - Memória do kernel
 - Ferramentas auxiliares: ex.: dtrace

Depuração, geração e boot do SO

- “Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.”
(Kernighan’s Law)

Depuração, geração e boot do SO

- SOs são projetados para executar em qualquer máquina de uma determinada classe
 - Devem ser configurados para cada tal
 - SYSGEN: programa para obter informação de configuração do hardware do sistema
 - Quantas e quais CPUs, como formatar disco, quantidade de memória (acesso ilegal à memória), dispositivos disponíveis e seus tipos, quais opções do SO a serem instaladas

Depuração, geração e boot do SO

- Boot do sistema: inicialização do computador através da carga do kernel
 - Processo feito pelo programa de bootstrap: código armazenado em ROM que localiza kernel, carrega na memória e inicia sua execução